Graph Algorithms Reference

22 tested Python templates. V = nodes, E = edges, R·C = grid cells. Page 1 is the complexity summary; definitions follow.

Algorithm	Time	Space
Build adjacency list	O(V + E)	O(V + E)
BFS: hop distances	O(V + E)	O(V)
DFS: recursive	O(V + E)	O(V)
DFS: iterative	O(V + E)	O(V)
Number of islands (grid DFS)	O(R·C)	O(R·C)
Count connected components	O(V + E)	O(V)
Cycle detection: undirected	O(V + E)	O(V)
Cycle detection: directed	O(V + E)	O(V)
Topological sort: Kahn's	O(V + E)	O(V)
Topological sort: DFS	O(V + E)	O(V)
Bipartite check	O(V + E)	O(V)
Multi-source BFS (rotting oranges)	O(R·C)	O(R·C)
0-1 BFS	O(V + E)	O(V)
Dijkstra	O((V + E) log V)	O(V + E)
Bellman-Ford	O(V · E)	O(V)
Cheapest flights, K stops	O(K · E)	O(V)
Floyd-Warshall	$O(V^3)$	$O(V^2)$
Union-Find (size + path compression)	O(α(n)) amortised	O(n)
Kruskal's MST	O(E log E)	O(V)
Prim's MST	O(E log V)	O(V + E)
Strongly connected components: Kosaraju	O(V + E)	O(V + E)
Bridges: Tarjan	O(V + E)	O(V)

Graph basics

1. Build adjacency list

TIME O(V + E) SPACE O(V + E)

Pre-initialise every node so isolated nodes exist. Weighted: append (v, w).

```
def build_adj(n, edges, directed=False):
    adj = {i: [] for i in range(n)}
    for u, v in edges:
        adj[u].append(v)
        if not directed:
            adj[v].append(u)
    return adj
```

Traversals

2. BFS: hop distances

TIME O(V + E) SPACE O(V)

Mark on ENQUEUE. First arrival = shortest path in an unweighted graph.

```
def bfs(adj, start):
    dist = {start: 0}
    q = deque([start])
    while q:
        node = q.popleft()
        for nb in adj[node]:
            if nb not in dist:          # mark on ENQUEUE
                dist[nb] = dist[node] + 1
                q.append(nb)
    return dist
```

3. DFS: recursive

TIME O(V + E) SPACE O(V)

Mark BEFORE recursing. Python recursion limit is 1000.

```
def dfs(adj, start, visited=None):
    if visited is None:
        visited = set()
    visited.add(start)                  # mark BEFORE recursing
    for nb in adj[start]:
        if nb not in visited:
            dfs(adj, nb, visited)
    return visited
```

4. DFS: iterative

TIME O(V + E) SPACE O(V)

Push reversed() to match the recursive order.

```
def dfs_iterative(adj, start):
    visited, order = {start}, []
    stack = [start]
    while stack:
        node = stack.pop()
        order.append(node)
        for nb in reversed(adj[node]):
            if nb not in visited:
                visited.add(nb)
                stack.append(nb)
    return order
```

5. Number of islands (grid DFS)

TIME O(R·C) SPACE O(R·C)

Outer loop finds land, sink() floods the island.

```
def num_islands(grid):
    rows, cols = len(grid), len(grid[0])
    def sink(r, c):
        if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] != "1":
            return
        grid[r][c] = "0"                # mark BEFORE recursing
        for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
            sink(r + dr, c + dc)
    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == "1":
                count += 1
                sink(r, c)
    return count
```

Components and cycles

6. Count connected components

TIME O(V + E) SPACE O(V)

With DSU: n – (number of successful unions).

```
def count_components(n, edges):
    adj = build_adj(n, edges)
    visited, count = set(), 0
    for node in range(n):
        if node not in visited:
            count += 1
            dfs(adj, node, visited)
    return count
```

7. Cycle detection: undirected

TIME O(V + E) SPACE O(V)

Watch the DOOR: a visited neighbour that is not the parent.

```
def has_cycle_undirected(n, edges):
    adj = build_adj(n, edges)
    visited = set()
    def dfs(node, parent):
        visited.add(node)
        for nb in adj[node]:
            if nb == parent:
                continue                # the U-turn, not a cycle
            if nb in visited or dfs(nb, node):
                return True
        return False
    return any(node not in visited and dfs(node, -1) for node in range(n))
```

8. Cycle detection: directed

TIME O(V + E) SPACE O(V)

Watch the STACK: an edge to a GRAY node.

```
def has_cycle_directed(n, edges):
    adj = build_adj(n, edges, directed=True)
    WHITE, GRAY, BLACK = 0, 1, 2
    color = [WHITE] * n
    def dfs(node):
        color[node] = GRAY
        for nb in adj[node]:
            if color[nb] == GRAY:
                return True             # edge back onto the current path
            if color[nb] == WHITE and dfs(nb):
                return True
        color[node] = BLACK
        return False
    return any(color[node] == WHITE and dfs(node) for node in range(n))
```

Ordering and colouring

9. Topological sort: Kahn's

TIME O(V + E) SPACE O(V)

Admission: indegree hits 0. Verdict: len(order) == n, else cycle.

```
def topo_kahn(n, edges):
    adj = build_adj(n, edges, directed=True)
    indegree = [0] * n
    for u, v in edges:
        indegree[v] += 1
    pool = deque(i for i in range(n) if indegree[i] == 0)
    order = []
    while pool:
        node = pool.popleft()
        order.append(node)
        for nb in adj[node]:
            indegree[nb] -= 1
            if indegree[nb] == 0:       # admission test
                pool.append(nb)
    return order if len(order) == n else []   # verdict: short lineup = cycle
```

10. Topological sort: DFS

TIME O(V + E) SPACE O(V)

Reverse post-order. Assumes a DAG.

```
def topo_dfs(n, edges):
    adj = build_adj(n, edges, directed=True)
    visited, post = set(), []
    def dfs(node):
        visited.add(node)
        for nb in adj[node]:
            if nb not in visited:
                dfs(nb)
        post.append(node)               # finished = everything after me is placed
    for node in range(n):
        if node not in visited:
            dfs(node)
    return post[::-1]
```

11. Bipartite check

TIME O(V + E) SPACE O(V)

Neighbour gets 1 − colour. Same colour across an edge = odd cycle.

```
def is_bipartite(adj):
    color = {}
    for start in adj:
        if start in color:
            continue
        color[start] = 0
        q = deque([start])
        while q:
            node = q.popleft()
            for nb in adj[node]:
                if nb not in color:
                    color[nb] = 1 - color[node]
                    q.append(nb)
                elif color[nb] == color[node]:
                    return False
    return True
```

Shortest paths

12. Multi-source BFS (rotting oranges)

TIME O(R·C) SPACE O(R·C)

Seed every source at time 0. Snapshot len(q) per level.

```
def oranges_rotting(grid):
    rows, cols = len(grid), len(grid[0])
    q, fresh = deque(), 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2:
                q.append((r, c))
            elif grid[r][c] == 1:
                fresh += 1
    minutes = 0
    while q and fresh:
        for _ in range(len(q)):         # one ring = one minute
            r, c = q.popleft()
            for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
                nr, nc = r + dr, c + dc
                if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1:
                    grid[nr][nc] = 2
                    fresh -= 1
                    q.append((nr, nc))
        minutes += 1
    return minutes if fresh == 0 else -1
```

13. 0-1 BFS

TIME O(V + E) SPACE O(V)

Weight 0 → appendleft, weight 1 → append.

```
def zero_one_bfs(adj, start):
    # adj[u] = [(v, w)] with w in {0, 1}
    dist = {start: 0}
    dq = deque([start])
    while dq:
        node = dq.popleft()
        for nb, w in adj[node]:
            if dist[node] + w < dist.get(nb, float("inf")):
                dist[nb] = dist[node] + w
                if w == 0:
                    dq.appendleft(nb)
                else:
                    dq.append(nb)
    return dist
```

14. Dijkstra

TIME O((V + E) log V) SPACE O(V + E)

Non-negative weights only. Skip stale heap entries.

```
def dijkstra(adj, start):
    # adj[u] = [(v, w)], all w >= 0
    dist = {start: 0}
    heap = [(0, start)]
    while heap:
        d, node = heapq.heappop(heap)
        if d > dist[node]:
            continue                    # stale entry, already improved
        for nb, w in adj[node]:
            nd = d + w
            if nd < dist.get(nb, float("inf")):
                dist[nb] = nd
                heapq.heappush(heap, (nd, nb))
    return dist
```

15. Bellman-Ford

TIME O(V · E) SPACE O(V)

n – 1 rounds. Still improving on round n = negative cycle.

```
def bellman_ford(n, edges, start):
    # edges = [(u, v, w)], w may be negative
    INF = float("inf")
    dist = [INF] * n
    dist[start] = 0
    for _ in range(n - 1):
        for u, v, w in edges:
            if dist[u] != INF and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    for u, v, w in edges:
        if dist[u] != INF and dist[u] + w < dist[v]:
            return None                 # negative cycle reachable
    return dist
```

16. Cheapest flights, K stops

TIME $O(K \cdot E)$ SPACE $O(V)$

K + 1 rounds, relax from a COPY of the last round.

```python
def cheapest_k_stops(n, flights, src, dst, k):
    INF = float("inf")
    dist = [INF] * n
    dist[src] = 0
    for _ in range(k + 1):              # k stops = k+1 edges
        prev = dist[:]                  # relax from LAST round only
        for u, v, w in flights:
            if prev[u] + w < dist[v]:
                dist[v] = prev[u] + w
    return -1 if dist[dst] == INF else dist[dst]
```

17. Floyd-Warshall

TIME $O(V^3)$ SPACE $O(V^2)$

k loop OUTSIDE. Negative cycle: dist[i][i] < 0.

```python
def floyd_warshall(n, edges):
    INF = float("inf")
    dist = [[INF] * n for _ in range(n)]
    for i in range(n):
        dist[i][i] = 0
    for u, v, w in edges:
        dist[u][v] = min(dist[u][v], w)
    for k in range(n):                  # k OUTSIDE: "allowed to pass through 0..k"
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
    return dist
```

Union-Find and MST

18. Union-Find (size + path compression)

TIME $O(\alpha(n))$ amortised SPACE $O(n)$

Small joins large; winner absorbs loser's size; link LEADERS.

```python
class UnionFind:
    def __init__(self, n):
        self.leader = list(range(n))
        self.size = [1] * n
    def find(self, x):
        if self.leader[x] != x:
            self.leader[x] = self.find(self.leader[x])   # path compression
        return self.leader[x]
    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False                # same leader twice = redundant edge
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        self.leader[rb] = ra            # small joins large
        self.size[ra] += self.size[rb]  # winner absorbs the LOSER's roster
        return True
```

19. Kruskal's MST

TIME O(E log E) SPACE O(V)

Sort the menu. Union is the bouncer. Close at n − 1.

```
def kruskal(n, edges):
    # edges = [(w, u, v)]
    uf = UnionFind(n)
    total, used = 0, 0
    for w, u, v in sorted(edges):       # sort the menu
        if uf.union(u, v):              # union is the bouncer
            total += w
            used += 1
            if used == n - 1:           # close at n-1
                break
    return total if used == n - 1 else -1
```

20. Prim's MST

TIME O(E log V) SPACE O(V + E)

Heap key is the EDGE weight, not the total distance.

```
def prim(n, adj):
    # adj[u] = [(v, w)]
    in_tree = set()
    heap = [(0, 0)]
    total = 0
    while heap and len(in_tree) < n:
        w, node = heapq.heappop(heap)
        if node in in_tree:
            continue
        in_tree.add(node)
        total += w
        for nb, nw in adj[node]:
            if nb not in in_tree:
                heapq.heappush(heap, (nw, nb))
    return total if len(in_tree) == n else -1
```

Advanced

21. Strongly connected components: Kosaraju

TIME O(V + E) SPACE O(V + E)

Finish order, reverse the graph, DFS in reverse finish order.

```
def kosaraju(n, edges):
    adj = build_adj(n, edges, directed=True)
    radj = build_adj(n, [(v, u) for u, v in edges], directed=True)
    visited, finish = set(), []
    def dfs1(node):
        visited.add(node)
        for nb in adj[node]:
            if nb not in visited:
                dfs1(nb)
        finish.append(node)
    for i in range(n):
        if i not in visited:
            dfs1(i)
    visited.clear()
    sccs = []
    def dfs2(node, comp):
        visited.add(node)
        comp.append(node)
        for nb in radj[node]:
            if nb not in visited:
                dfs2(nb, comp)
    for node in reversed(finish):
        if node not in visited:
            comp = []
            dfs2(node, comp)
            sccs.append(comp)
    return sccs
```

22. Bridges: Tarjan

TIME O(V + E) SPACE O(V)

Bridge if low[child] > disc[parent]. Articulation point uses ≥.

```
def bridges(n, edges):
    adj = build_adj(n, edges)
    disc, low, out = [-1] * n, [0] * n, []
    timer = [0]
    def dfs(node, parent):
        disc[node] = low[node] = timer[0]
        timer[0] += 1
        for nb in adj[node]:
            if nb == parent:
                continue
            if disc[nb] == -1:
                dfs(nb, node)
                low[node] = min(low[node], low[nb])
                if low[nb] > disc[node]:
                    out.append((node, nb))
            else:
                low[node] = min(low[node], disc[nb])
    for i in range(n):
        if disc[i] == -1:
            dfs(i, -1)
    return out
```